Movie Recommendation Systems Project

Matthew Rivers | DSC 201 | Fall 2026

1. Project Description

Dataset Overview

In this project, we will be building multiple different types of movie recommendation systems. This project will use the **HetRec 2011 MovieLens 2k** dataset, released by GroupLens for the 2nd International Workshop on Information Heterogeneity and Fusion in Recommender Systems (HetRec 2011). It extends the MovieLens 10M dataset by linking each movie to IMDb and Rotten Tomatoes, and by providing additional information about the movies, such as the cast, director, and genre. The dataset contains **855,598 ratings** on a 0.5-5 star scale from **2,113 users**, applied to **10,197 movies**. The dataset is available at https://grouplens.org/datasets/hetrec-2011/

Features

The dataset contains 12 separate `.dat` files. Each one contains different information about the movies, users, and ratings. The files will be explained more in detail in *Section 3: Data Loading*. The actual features we will be using for our recommendation systems are:

Content Based Filtering Features:

These features will be used to combine into one string for each movie, which will allow us to use a vectorizer.

- `genres` - The genre(s) of the movie (from `movie_genres.dat`)
- `director` - the director of the movie (from `movie_directors.dat`)
- `actors` - the top 5 most billed actors per IMDb (from `movie_actors.dat`)
- `tags` - the tags assigned to the movie (from `tags.dat` and `movie_tags.dat`)

Collaborative Filtering (SVD) Features:

These features detail the ratings given by each user to each movie, which will allow us to use SVD to predict what ratings a user would give to a movie they have not yet rated.

- `userID` - the user id (from `user_ratedmovies.dat`)
- `movieID` - the movie id (from `user_ratedmovies.dat`)
- `rating` - the rating given by the user to the movie (from `user_ratedmovies.dat`)

Hybrid Model Features:

The hybrid model uses both features from content based filtering and collaborative filtering

Miscellaneous Features:

- `title_year` - this is a custom feature made of the title and the year, displays as 'Toy Story (1995)'. It was created as many movies have the same title but were released in separate years.
- `num_ratings` - this feature is the number of ratings a movie has received. It will be used to filter out movies with too few ratings for the collaborative filtering (SVD) model for reliability purposes.
- `year` - the year the movie was released.

Project Goal

The goal of this project is to build and compare three recommendation systems:

- **Content-Based Filtering**: This system will recommend movies based on the content of the movies, such as genre, director, and cast. It will use the movie features to find other movies similar to a given movie.
- **Collaborative Filtering (SVD)**: This system will recommend movies based on predictions of how a user would rate a movie by learning from the ratings of other users. It uses Singular Value Decomposition (SVD) to find latent factors that explain the ratings of users and movies.
- **Hybrid Filtering**: This system will combine the content-based and collaborative filtering approaches. It will use both the movie features and the ratings of users to recommend movies.

The models will then be evaluated in two ways:

1. RMSE will be used to choose the number of latent features for the collaborative filtering (SVD) model and to measure how accurately it predicts ratings.

2. All three models will be compared using hit rate @ 10 against a simple popularity baseline. We will also inspect recommendations for familiar movies to make sure they make sense.

2. Imports

Import Libraries

We will start by importing the necessary libraries for all three recommender systems. We will be using the following libraries:

- `pandas` loading/cleaning the data
- `numpy` used in SVD
- `sklearn` for the vectorizer and cosine similarity
- `scipy` used in SVD

In [132…

```
# importing all libraries
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from scipy.sparse.linalg import svds
```

3. Data Loading

File Names and Descriptions

Now we need to load the data that we will be using for our recommender systems. The dataset comes as 12 separate `.dat` files and a readme file. The files we will be using are:

- `movies.dat` : movies in the database.
- `movie_genres.dat` : genres of the movies in the database.
- `movie_directors.dat` : directors of the movies in the database.
- `movie_actors.dat` : actors/actresses of the movies in the database. A ranking is given to the actors/actresses based on their importance in the movie (based on IMDb). The first actor/actress listed is the most important, and the last is the least important.
- `tags.dat` : set of tags available in the database. The tags are used to describe the movies and can be used to find similar movies.
- `movie_tags.dat` : tags assigned to the movies in the database, as well as the number of times the tag has been assigned to the movie.
- `user_ratedmovies.dat` : ratings of the movies by the users in the database. The ratings are on a 0.5-5 star scale, with 0.5 being the lowest and 5 being the highest.

Note that as listed we are only using 7 out of the 12 files in the dataset. The other files are not needed for our recommendation systems. The files we are not using are:

- `movie_countries.dat` : countries of the movies in the database. We will not be using this file as country adds little value to our recommendation systems compared to the other features we are using.
- `movie_locations.dat` : filming locations of the movies in the database. We will not be using this file as the location of where a movie was filmed doesn't tell us much about the movie itself and is not a good indicator of whether a user will like the movie or not.
- `user_ratedmovies-timestamps.dat` : same information as `user_ratedmovies.dat` but with timestamps of when the user rated the movie. We

will not be using this file as we are not concerned with when a user rated a movie, only what they rated it.

- `user_taggedmovies.dat` and `user_taggedmovies-timestamps.dat`: tags assigned to the movies by the users in the database, as well as the timestamps of when the user assigned the tag. We will not be using these files as it's redundant considering we are already using `movie_tags.dat`.

All of the above described files were detailed in the readme file that came with the dataset. The readme file can be found at https://grouplens.org/datasets/hetrec-2011/ and is titled `README.txt`.

Loading the Data

We will take a few steps to ensure proper loading of the data.

- All files are tab-separated, so we will use `sep='\t'`.
- Some titles and names contain special characters, so we will use `encoding='latin-1'`
- Missing values are represented by `\N`, so we will use `na_values='\\N'` to tell pandas to treat these as NaN values.

In [133…
```
# acessing the folder containing the files
data_path = 'C:/Users/river/OneDrive/Student/University of Tampa/Junior/Fall Semest

# loading each file as a pandas df
movies = pd.read_csv(data_path + 'movies.dat', sep='\t', encoding='latin-1', na_val
genres = pd.read_csv(data_path + 'movie_genres.dat', sep='\t', encoding='latin-1',
directors = pd.read_csv(data_path + 'movie_directors.dat', sep='\t', encoding='lati
actors = pd.read_csv(data_path + 'movie_actors.dat', sep='\t', encoding='latin-1',
movie_tags = pd.read_csv(data_path + 'movie_tags.dat', sep='\t', encoding='latin-1'
tags = pd.read_csv(data_path + 'tags.dat', sep='\t', encoding='latin-1', na_values=
ratings = pd.read_csv(data_path + 'user_ratedmovies.dat', sep='\t', encoding='latin
```

Shapes/Sizes

Now, we will display the shape of each of the files we are using to get a better understanding of the data we are working with.

In [134…
```
# printing shapes using a for loop
files_names = {'movies': movies, 'genres': genres, 'directors': directors, 'actors'
for name, files in files_names.items():
    print(f"{name} shape: {files.shape[0]} rows, {files.shape[1]} columns")
```

```
movies shape: 10197 rows, 21 columns
genres shape: 20809 rows, 2 columns
directors shape: 10155 rows, 3 columns
actors shape: 231742 rows, 4 columns
movie_tags shape: 51795 rows, 3 columns
tags shape: 13222 rows, 2 columns
ratings shape: 855598 rows, 9 columns
```

Previews

Now, we will display the head of each of the files to get a sense of what the data looks like.

Movies

In [135…
```
movies.head()
```

Out[135…

	id	title	imdbID	spanishTitle	imdbPictureURL	year	
0	1	Toy story	114709	Toy story (juguetes)	http://ia.media-imdb.com/images/M/MV5BMTMwNDU0...	1995	
1	2	Jumanji	113497	Jumanji	http://ia.media-imdb.com/images/M/MV5BMzM5NjE1...	1995	
2	3	Grumpy Old Men	107050	Dos viejos gruñones	http://ia.media-imdb.com/images/M/MV5BMTI5MTgy...	1993	
3	4	Waiting to Exhale	114885	Esperando un respiro	http://ia.media-imdb.com/images/M/MV5BMTczMTMy...	1995	
4	5	Father of the Bride Part II	113041	Vuelve el padre de la novia (Ahora también abu...	http://ia.media-imdb.com/images/M/MV5BMTg1NDc2...	1995	father_o

5 rows × 21 columns

The `movies` file contains basic information and critic scoring details about the movies in the database.

Genres

In [136…
```
genres.head()
```

Out[136…

	movieID	genre
0	1	Adventure
1	1	Animation
2	1	Children
3	1	Comedy
4	1	Fantasy

The `genres` file contains the genres of each movie.

Directors

In [137…

```
directors.head()
```

Out[137…

	movieID	directorID	directorName
0	1	john_lasseter	John Lasseter
1	2	joe_johnston	Joe Johnston
2	3	donald_petrie	Donald Petrie
3	4	forest_whitaker	Forest Whitaker
4	5	charles_shyer	Charles Shyer

The `directors` file contains the directors of each movie.

Actors

In [138…

```
actors.head()
```

Out[138…

	movieID	actorID	actorName	ranking
0	1	annie_potts	Annie Potts	10
1	1	bill_farmer	Bill Farmer	20
2	1	don_rickles	Don Rickles	3
3	1	erik_von_detten	Erik von Detten	13
4	1	greg-berg	Greg Berg	17

The `actors` file contains the actors/actresses of each movie, and their importance.

Tags

In [139…

```
tags.head()
```

Out[139…

	id	value
0	1	earth
1	2	police
2	3	boxing
3	4	painter
4	5	whale

The `tags` file contains the set of tags available in the database.

Movie Tags

In [140…
```
movie_tags.head()
```

Out[140…

	movieID	tagID	tagWeight
0	1	7	1
1	1	13	3
2	1	25	3
3	1	55	3
4	1	60	1

The `movie_tags` file contains the tags assigned to each movie as well as the weight in which the tag has been assigned to the movie.

Ratings

In [141…
```
ratings.head()
```

Out[141…

	userID	movieID	rating	date_day	date_month	date_year	date_hour	date_minute	da
0	75	3	1.0	29	10	2006	23	17	
1	75	32	4.5	29	10	2006	23	23	
2	75	110	4.0	29	10	2006	23	30	
3	75	160	2.0	29	10	2006	23	16	
4	75	163	4.0	29	10	2006	23	29	

The `ratings` file contains the ratings of the movies by the users.

4. Data Cleaning and Preprocessing

Before we can build any of our recommendation systems, we need to clean and preprocess the data. This includes:

- Ensuring each movie is represented by one row with one combined description (important for content-based filtering)
- Ensuring that the ratings table contains only the most reliable ratings (important for collaborative filtering)
- Ensuring there are no missing values, duplicates, or other issues that would cause problems for our models.

Movies

We will start by cleaning the `movies` file, keeping only the columns necessary for the recommender systems. The columns we will keep are:

- `id`
- `title`
- `year`

`id` and `title` are necessary for movie identification and display, while `year` is important for movies with the same title but different release years. The remaining columns (image URL's, spanish titles, IMDb/Rotten Tomatoes information, etc.) will be dropped as they are not needed for our recommender systems. The critics ratings and scores will not be used as they are not a good indicator of whether a user will like a movie or not, they only define the reception of the movie rather than its actual content. The ratings of the users in the `ratings` file will be used instead as they are a better indicator.

In [142…
```
movies = movies[['id', 'title', 'year']].copy()
```

We will also rename `id` to `movieID`, so that it matches the other files we will be using. This will make it easier to merge the files later on.

In [143…
```
movies = movies.rename(columns={'id': 'movieID'})
```

Next, we need to check for missing values and check the data types of each column.

In [144…
```
print(movies.isnull().sum())
print()
print(movies.dtypes)
```

```
movieID    0
title      0
year       0
dtype: int64

movieID     int64
title      object
year        int64
dtype: object
```

We see that there are no missing values. The `movieID` column and the `year` column are both `int64` data types, and then title column is an `object` data type.

We also need to make sure that every movie has a unique name. We will check for duplicates by creating a new column `title_year` to rename movies with their year (ex. if title is Toy Story and year is 1995, title_year = Toy Story (1995). This will help to find movies that share the same title but have different release years.

In [145…
```
movies['title_year'] = movies['title'] + ' (' + movies['year'].astype(str) + ')'
print(f"Duplicate titles: {movies['title_year'].duplicated().sum()}")
```

```
Duplicate titles: 775
```

There are 775 duplicate `title_year` values, meaning there are 775 movies that share the same title and year. This is an issue that needs further investigation. We will check the original `movies` file to check the root issue, whether it's an issue on our end or the datasets end. We can start by looking at which movie has the most repeated entries.

In [146…
```
# the most repeated title_year value
print(movies['title_year'].value_counts().head(1))
```

```
title_year
Hamlet (1996)    6
Name: count, dtype: int64
```

Now let's examine Hamlet (1996) to see what's causing the duplicates. We will look at the original `movies` file to see if there are any differences in the rows for Hamlet (1996).

In [147…
```
# reloading 'movies' but as the original version
raw_movies = pd.read_csv(data_path + 'movies.dat', sep='\t', encoding='latin-1', na

raw_movies[raw_movies['title'] == 'Hamlet']
```

Out[147...

	id	title	imdbID	spanishTitle	imdbPictureURL	year	
1284	1411	Hamlet	116477	Hamlet	http://ia.media-imdb.com/images/M/MV5BMTM5NjMz...	1996	10
1735	1941	Hamlet	116477	Hamlet	http://ia.media-imdb.com/images/M/MV5BMTM5NjMz...	1996	10
2589	2820	Hamlet	116477	Hamlet	http://ia.media-imdb.com/images/M/MV5BMTM5NjMz...	1996	10
3319	3598	Hamlet	116477	Hamlet	http://ia.media-imdb.com/images/M/MV5BMTM5NjMz...	1996	10
3441	3723	Hamlet	116477	Hamlet	http://ia.media-imdb.com/images/M/MV5BMTM5NjMz...	1996	10
9986	61246	Hamlet	116477	Hamlet	http://ia.media-imdb.com/images/M/MV5BMTM5NjMz...	1996	10

6 rows × 21 columns

All 6 "Hamlet (1996)" entries have the same `imdbID`, `rtID` and identical Rotten Tomatoes scores. They do however have different MovieLens IDs (`id` column). This means that the duplicates are an issue with the dataset itself; perhaps the matching of the movies to the IMDb and Rotten Tomatoes databases was done incorrectly, or perhaps the movies were added to the MovieLens database multiple times. This also may or may not carry over to the cast, director, and other information within other files as well.

Since we can't recover the true identity of the duplicated movies, we should only keep the ones with the most user ratings, as they are the most reliable. We also compare titles in lowercase, in case there are duplicates due to capitalization differences.

In [148...

```
# coutning how many ratings each movie has
rating_counts = ratings.groupby('movieID').size().rename('num_ratings')

# adding to movies
movies = movies.merge(rating_counts, on='movieID', how = 'left')
# making sure movies without ratings have 0
movies['num_ratings'] = movies['num_ratings'].fillna(0).astype(int)

# case insensitive duplicate check
movies['title_lower'] = movies['title_year'].str.lower()

# keeping only the movies with the most ratings for each title_year
movies = movies.sort_values('num_ratings', ascending=False)
movies = movies.drop_duplicates(subset='title_lower', keep='first')
movies = movies.drop(columns='title_lower').sort_values('movieID').reset_index(drop

# checking to make sure there are no more duplicate title_year values
print(f"Movies Remaining: {movies.shape[0]}")
```

```
# .lower to make sure it is case insensitive
print(f"Duplicate title_year values remaining: {movies['title_year'].str.lower().du
```

```
Movies Remaining: 9421
Duplicate title_year values remaining: 0
```

In [149…
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9421 entries, 0 to 9420
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   movieID      9421 non-null   int64 
 1   title        9421 non-null   object
 2   year         9421 non-null   int64 
 3   title_year   9421 non-null   object
 4   num_ratings  9421 non-null   int32 
dtypes: int32(1), int64(2), object(2)
memory usage: 331.3+ KB
```

Now there are no more duplicate `title_year` values, meaning that every movie is only represented once. There are a total of 9,421 unique movies. We can see the final preview of `movies` below.

In [150…
```
movies.head()
```

Out[150…

	movieID	title	year	title_year	num_ratings
0	1	Toy story	1995	Toy story (1995)	1263
1	2	Jumanji	1995	Jumanji (1995)	765
2	4	Waiting to Exhale	1995	Waiting to Exhale (1995)	45
3	5	Father of the Bride Part II	1995	Father of the Bride Part II (1995)	225
4	6	Heat	1995	Heat (1995)	586

Genres

Now we need to clean the `genres` file. We can recall that the `genres` file contains the id of the movie and the genre(s) of the movies. However, each genre listed for a movie is in a seperate row, meaning that a movie with multiple genres will have multiple rows in the `genres` file. This is an issue for our content-based filtering model, as we want each movie to be represented by one row with one combined description. We will need to combine the genres for each movie into one row. We will begin by counting the number of unique genres within the `genres` file, as well as how many times each genre appears.

In [151…
```
print(f"Number of unique genres: {genres['genre'].nunique()}")
genres['genre'].value_counts()
```

```
Number of unique genres: 20
```

Out[151…

```
genre
Drama          5076
Comedy         3566
Thriller       1664
Romance        1644
Action         1445
Crime          1086
Adventure      1003
Horror          978
Sci-Fi          740
Fantasy         535
Children        519
Mystery         497
War             494
Documentary     430
Musical         421
Animation       279
Western         261
Film-Noir       145
IMAX             25
Short             1
Name: count, dtype: int64
```

We can see that there are 20 unique `genres`, and certain ones are more popular than others. Drama, Comedy, and Thriller are the most popular. One thing that needs fixing however is some of the `genres` contain special characters that may cause issues with our Vectorizer later on, specifically the hyphenated ones (ex. Film-Noir, Sci-Fi). We will need to remove the hyphens from these to avoid any issues later on, replacing them with a '_'.

In [152…

```
# replacing hyphens with _
genres['genre'] = genres['genre'].str.replace('-', '_')
```

Now, we need to combine all the `genres` for each movie into one string.

In [153…

```
movie_genres = genres.groupby('movieID')['genre'].apply(lambda x: ' '.join(x)).rese
movie_genres = movie_genres.rename(columns={'genre': 'genres'})
movie_genres.head()
```

Out[153…

	movieID	genres
0	1	Adventure Animation Children Comedy Fantasy
1	2	Adventure Children Fantasy
2	3	Comedy Romance
3	4	Comedy Drama Romance
4	5	Comedy

Now, as we can see in the preview above, each movies `genre` is now represented by one row of genres tagged in the data. This makes it easier to use as a feature as each `movieID`

only has one row insetad of multiple. The final step is to merge the `movie_genres` file with the `movies` file, so that each movie has its `genres` included in the same row.

In [154…
```
# merging onto movies
movies = movies.merge(movie_genres, on = 'movieID', how = 'left')
# filling nans with empty string
movies['genres'] = movies['genres'].fillna('')
# ensuring all movies have at least one genre
print(f"Movies with no genre: {movies[movies['genres'] == ''].shape[0]}")
movies.head()
```

```
Movies with no genre: 0
```

Out[154…

	movieID	title	year	title_year	num_ratings	genres
0	1	Toy story	1995	Toy story (1995)	1263	Adventure Animation Children Comedy Fantasy
1	2	Jumanji	1995	Jumanji (1995)	765	Adventure Children Fantasy
2	4	Waiting to Exhale	1995	Waiting to Exhale (1995)	45	Comedy Drama Romance
3	5	Father of the Bride Part II	1995	Father of the Bride Part II (1995)	225	Comedy
4	6	Heat	1995	Heat (1995)	586	Action Crime Thriller

Now, the `movies` file contains the genres of each movie, as well as the `movieID`, `title`, `year`. `title_year`, and `num_ratings`.

Directors

The `directors` file is structured similarly to the `genres` file, with a `movieId`, `directorID`, and `directorName`. We will take the same steps to fix any director names with special characters or spaces like how we did with genres. First let's check missing values. and check for any movies with multiple directors.

In [155…
```
print(directors.isnull().sum())
print()
print(f"Movies with more than one director: {directors['movieID'].duplicated().sum(
```

```
movieID         0
directorID      0
directorName    0
dtype: int64

Movies with more than one director: 0
```

There are no missing values, and no movie has more than one director. The data lists only one director per movie as well. Any spaces within director names may cause issues with our Vectorizer later on however. So, we will compare the number of unique `directorNames` and unique `directorIDs`, to find any issues.

In [156…
```
print(f"Number of unique directors: {directors['directorName'].nunique()}")
print(f"Number of unique directorIDs: {directors['directorID'].nunique()}")
```

```
Number of unique directors: 4053
Number of unique directorIDs: 4060
```

4,060 unique `directorIDs` against 4,053 unique `directorNames`. This means the same director may appear under multiple `directorIDs`. So, we will use `directorName` (after cleaning it) instead of `directorID`, as the names are consistent and the IDs are not. We will create a function to clean the director names (which can be used later for actor names and other names), removing spaces and/or special characters.

In [157…
```
# function to clean names into one string
# this is what directorID is supposed to be, but is done poorly in the data
# was constantly running into columns/features with random characters, so decided b
# below is the function i originally had, before I googled and found '.isalnum()' w

# def clean_name(name):
#     # in case any name has the word 'Director' or 'Actor' in it
#     name = name.replace('Director ', '')
#     name = name.replace('Actor ', '')
#     # removing periods, apostrophes, parenthesis, dashes, spaces, any possible sp
#     name = name.replace('.', '')
#     name = name.replace("'", '')
#     name = name.replace('(', '')
#     name = name.replace(')', '')
#     name = name.replace('-', '')
#     name = name.replace(' ', '')
#     name = name.replace('"', '')
#     name = name.replace('[', '')
#     name = name.replace(']', '')
#     name = name.replace('!', '')
#     name = name.replace('?', '')
#     name = name.replace(',', '')
#     name = name.replace('&', '')
#     name = name.replace('@', '')
#     name = name.replace('#', '')
#     # making all lowercase
#     return name.lower()

# now here is new / better approach using .isalnum()
def clean_name(name):
    # still keeping this in case any name has the word 'Director' / 'Actor' in it
    name = name.replace('Director ', '')
    name = name.replace('Actor ', '')
    # for loop to append each number or character to the string (cleaned)
    cleaned = ''
    for character in name:
```

```
        if character.isalnum():
            cleaned += character
    return cleaned.lower()
```

Now we will apply the `clean_name` function to every director name in the `directors` file. This will remove any spaces or special characters from the director names, making them easier to use as a feature.

In [158…

```
directors['directorName'] = directors['directorName'].apply(clean_name)
# validating
directors.head()
```

Out[158…

	movieID	directorID	directorName
0	1	john_lasseter	johnlasseter
1	2	joe_johnston	joejohnston
2	3	donald_petrie	donaldpetrie
3	4	forest_whitaker	forestwhitaker
4	5	charles_shyer	charlesshyer

Now, we can see the director names have been cleaned up and are ready to be used as a feature. The final step is to merge the `directors` file with the `movies` file.

In [159…

```
# merging onto movies
movies = movies.merge(directors[['movieID', 'directorName']], on = 'movieID', how =
# renaming directorName to director for clarity
movies = movies.rename(columns={'directorName': 'director'})

# movies without director get empty string
movies['director'] = movies['director'].fillna('')

# checking how many movies have no director
print(f"Movies with no director: {movies[movies['director'] == ''].shape[0]}")

# validating / preview
movies.head()
```

```
Movies with no director: 40
```

Out[159…

	movieID	title	year	title_year	num_ratings	genres	director
0	1	Toy story	1995	Toy story (1995)	1263	Adventure Animation Children Comedy Fantasy	johnlasseter
1	2	Jumanji	1995	Jumanji (1995)	765	Adventure Children Fantasy	joejohnston
2	4	Waiting to Exhale	1995	Waiting to Exhale (1995)	45	Comedy Drama Romance	forestwhitaker
3	5	Father of the Bride Part II	1995	Father of the Bride Part II (1995)	225	Comedy	charlesshyer
4	6	Heat	1995	Heat (1995)	586	Action Crime Thriller	michaelmann

As we can now see, director names have been added to the `movies` file. Also to note, there are 40 movies within the data with no director. The `movies` file now contains the `movieID`, `title`, `year`, `title_year`, `num_ratings`, `genres`, and `director` for each movie.

Actors

Next, we will clean the `actors` file. The file contains the `movieID`, `actorID`, `actorName`, and `ranking`. Remember that the `ranking` is based on the importance of the actor in the movie. We will explore how the ranking works (max's, min's, etc.) as well as clean the `actorName` features with the `clean_name` function we used for director names. We will also check for missing values.

In [160…

```
# checking missing values
print(actors.isnull().sum())
print()

# amount of actors per movie
print("Actors per movie:")
print(actors['movieID'].value_counts().describe())
print()

# ranking orders
print("Ranking:")
print(actors['ranking'].describe())
```

```
movieID      0
actorID      0
actorName    5
ranking      0
dtype: int64

Actors per movie:
count    10174.000000
mean        22.777865
std         17.560943
min          1.000000
25%         11.000000
50%         17.000000
75%         30.000000
max        220.000000
Name: count, dtype: float64

Ranking:
count    231742.000000
mean         18.522732
std          18.368837
min           1.000000
25%           6.000000
50%          13.000000
75%          25.000000
max         220.000000
Name: ranking, dtype: float64
```

We can see that there are no missing values for `movieID`, `actorID`, or `ranking`, however there are 5 missing values for `actorName`. We will drop these 5 entries entirely as they are not useful features with missing names, and would crash the `clean_name` function if applied.

There are 10,174 movies in the data that have actors listed, with around 22.7 actors per movie (comes from the mean). As for the rankings, there are a total of 231,742 rankings. The rankings start at 1 (as per the min, top-billed) and go up to the 220 (as per the max, lowest importance). The next step we will take is to only keep the top 5 actors from each movie, as they are the most important and telling feature.

In [161…

```python
# dropping the 5 rows with missing actorName
actors = actors.dropna(subset=['actorName'])

# only keeping the top 5 actors per movie
top_actors = actors.sort_values(['movieID', 'ranking']).groupby('movieID').head(5)

# validating no movie has more than 5 now
print(f"Maximum actors per movie: {top_actors['movieID'].value_counts().max()}")
top_actors.head()
```

```
Maximum actors per movie: 5
```

Out[161…

	movieID	actorID	actorName	ranking
22	1	tom_hanks	Tom Hanks	1
21	1	tim_allen	Tim Allen	2
2	1	don_rickles	Don Rickles	3
7	1	jim_varney	Jim Varney	4
23	1	wallace_shawn	Wallace Shawn	5

Now, we can apply `clean_name` to the `actorName` column, just like how we did with the directors. Then we will combine the top 5 actors into one string. Finally, we will merge the `actors` file with the `movies` file.

In [162…

```
# cleaning names
top_actors['actorName'] = top_actors['actorName'].apply(clean_name)

# combining top 5 actors into one string
movie_actors = top_actors.groupby('movieID')['actorName'].apply(lambda x: ' '.join(
movie_actors = movie_actors.rename(columns={'actorName': 'actors'})

# merging onto movies + filling nans with empty string
movies = movies.merge(movie_actors, on = 'movieID', how = 'left')
movies['actors'] = movies['actors'].fillna('')

# validating / preview
print(f"Movies with no actors: {movies[movies['actors'] == ''].shape[0]}")
movies.head()
```

```
Movies with no actors: 19
```

Out[162…

	movieID	title	year	title_year	num_ratings	genres	director	acto
0	1	Toy story	1995	Toy story (1995)	1263	Adventure Animation Children Comedy Fantasy	johnlasseter	tomhank timalle donrickle jimvarne wallace
1	2	Jumanji	1995	Jumanji (1995)	765	Adventure Children Fantasy	joejohnston	robinwilliam bonniehur kirstenduns bradleyp
2	4	Waiting to Exhale	1995	Waiting to Exhale (1995)	45	Comedy Drama Romance	forestwhitaker	whitneyhousto angelabasse lorettadevin lel
3	5	Father of the Bride Part II	1995	Father of the Bride Part II (1995)	225	Comedy	charlesshyer	stevemarti dianekeato martinsho kimberlywi
4	6	Heat	1995	Heat (1995)	586	Action Crime Thriller	michaelmann	alpacin robertdenir valkilme jonvoigh toms

As we can see now, there are only 19 movies without actors. The `movies` file now contains the `movieID`, `title`, `year`, `title_year`, `num_ratings`, `genres`, `director`, and `actors` for each movie.

Tags

Now we will clean the `tags` file. The file contains the id of the tag and the name/value. We will check for missing values, and check for any tags with special characters or spaces that may cause issues with our Vectorizer later on.

In [163…

```
# checking for missing values
print(tags.isnull().sum())
print()

# checking number of tags + unique tags (should match ideally)
print(f"Number of tags: {tags.shape[0]}")
print(f"Number of unique tags: {tags['value'].nunique()}")
print()
```

```
id       0
value    0
dtype: int64

Number of tags: 13222
Number of unique tags: 13222
```

We see that there are no missing values, and the number of unique tags matches the number of tags in the file. We can now apply the `clean_name` function to the tags.

In [164…
```
# testing clean_name on the tag values to see if any punctuation is left over
tags['value'] = tags['value'].apply(clean_name)

# checking for empty tags
print(f"Number of empty tags after cleaning: {(tags['value'] == '').sum()}")

# checking for unque tags after cleaning (tags that may have become identical after
print(f"Unique tags after cleaning: {tags['value'].nunique()}")
```

```
Number of empty tags after cleaning: 1
Unique tags after cleaning: 13101
```

We can drop the 1 leftover tag since it has no numbers/words and will not be useful. Since the amount of unique tags went from 13,222 to 13,101, this means that 121 tags merged with another after cleaning (ex. Comedy and comedy both became comedy). Now we will rename the `id` column to `tagID` to later match the `movie_tags` file.

In [165…
```
# dropping the leftover tag
tags = tags[tags['value'] != '']

# renaming
tags = tags.rename(columns={'id': 'tagID'})

# validating / preview
print(f"Tags remaining: {tags.shape[0]}")
tags.head()
```

```
Tags remaining: 13221
```

Out[165…

	tagID	value
0	1	earth
1	2	police
2	3	boxing
3	4	painter
4	5	whale

We're now back to 13,221 tags, and the id has been renamed to tagID. `tags` is now ready to be merged with the `movie_tags` file, which we will do in the `movie_tags` section.

Movie Tags

We will now clean the `movie_tags` file, following the same steps as we did with the `tags` file. Both are structured similarly; the `movie_tags` file currently contains `movieID`, `tagID`, and `tagWeight`. As always we will start by checking for missing values.

In [166…
```
# missing vals
print(movie_tags.isnull().sum())
print()

# movies with a tag
print("Movies with a tag:")
print(movie_tags['movieID'].nunique())
print()
```

```
movieID      0
tagID        0
tagWeight    0
dtype: int64

Movies with a tag:
7155
```

There are no missing values in the `movie_tags` file. 7,155 movies have a tag, and as for the weights, they are unimportant as all tags count equally. The next step is to merge the `movie_tags` file with the `tags` file to get the tag words. Then we can combine each movie's tags into one string, like how we did with the other files, and merge all of it onto `movies`.

In [167…
```
# movie_tags merged with tags
movie_tags = movie_tags.merge(tags, on='tagID', how='left')

# also dropping any rows that have a tagID that doesn't exist in the tags file (lik
movie_tags = movie_tags.dropna(subset=['value'])

# combining to one string
movie_tags_combined = movie_tags.groupby('movieID')['value'].apply(lambda x: ' '.jo
movie_tags_combined = movie_tags_combined.rename(columns={'value': 'tags'})

# merging onto movies + filling nans with empty string
movies = movies.merge(movie_tags_combined, on = 'movieID', how = 'left')
movies['tags'] = movies['tags'].fillna('')

# validatring / preview
print(f"Movies with no tags: {movies[movies['tags'] == ''].shape[0]}")
movies.head()
```

```
Movies with no tags: 2744
```

Out[167…

	movieID	title	year	title_year	num_ratings	genres	director	acto
0	1	Toy story	1995	Toy story (1995)	1263	Adventure Animation Children Comedy Fantasy	johnlasseter	tomhank timalle donrickle jimvarne wallace
1	2	Jumanji	1995	Jumanji (1995)	765	Adventure Children Fantasy	joejohnston	robinwilliam bonniehur kirstenduns bradleyp
2	4	Waiting to Exhale	1995	Waiting to Exhale (1995)	45	Comedy Drama Romance	forestwhitaker	whitneyhousto angelabasse lorettadevin lel
3	5	Father of the Bride Part II	1995	Father of the Bride Part II (1995)	225	Comedy	charlesshyer	stevemarti dianekeato martinsho kimberlywi
4	6	Heat	1995	Heat (1995)	586	Action Crime Thriller	michaelmann	alpacin robertdenir valkilme jonvoigh toms

There exist 2,744 movies with no tags, which while not optimal, there are still other features that are used in our models (genres, directors, etc.) that will help with the recommendations. As shown above, the `movies` file now contains the `movieID`, `title`, `year`, `title_year`, `num_ratings`, `genres`, `director`, `actors`, and `tags` for each movie.

Ratings

The `ratings` file contains quite a bit of information, including the `userID`, `movieID`, and the `rating` given by the user for that movie. It also includes the time info of the ratings, including day, month, year, hour, minute, second. These time features will not be used in our systems, as they don't add any value since they have nothing to do with the content of the movie or user. So, keeping only `userID`, `movieID`, and `rating`, we will check for missing values and clean the data.

In [168…

```
# keeping big 3
ratings = ratings[['userID', 'movieID', 'rating']]
```

```
# missing values
print(ratings.isnull().sum())
print()

# how many ratings are in the file
print(f"Ratings count: {ratings.shape[0]}")
```

```
userID     0
movieID    0
rating     0
dtype: int64

Ratings count: 855598
```

There are no missing values in the 3 columns we are keeping. In total, there currently exist 855,598 ratings. We can now move on to keep only the ratings for movies that are in our `movies` file, as we don't want to keep ratings for movies that we've already dropped.

In [169…
```
ratings = ratings[ratings['movieID'].isin(movies['movieID'])]
print(f"Ratings remaining after filtering to only include movies in the movies file
```

```
Ratings remaining after filtering to only include movies in the movies file: 827192
```

Now there are 827,192 ratings remaining, which are all of the ratings for movies actually in our `movies` file. Now we will check how many ratings each movie has, to determine how many to keep in our models.

In [170…
```
# how many ratings each movie has
print("Ratings per movie:")
print(ratings['movieID'].value_counts().describe())
```

```
Ratings per movie:
count    9344.000000
mean       88.526541
std       176.662328
min         1.000000
25%         7.000000
50%        22.000000
75%        80.000000
max      1670.000000
Name: count, dtype: float64
```

In [171…
```
# how many ratings each user has
print("Ratings per user:")
print(ratings['userID'].value_counts().describe())
```

```
Ratings per user:
count    2113.000000
mean      391.477520
std       402.303831
min        16.000000
25%       104.000000
50%       266.000000
75%       533.000000
max      3199.000000
Name: count, dtype: float64
```

The movies have a median of 22 ratings, with a mean of 88, indicating signifigant skews likely due to outliers (min = 1, max = 1670). As for the users, each user has at minimum 16 ratings, with a median of 266 and a mean of 391, indicating that the users have a pretty reliable amount of ratings to work with. The movies however, having a less reliable amount of ratings, will not suffice for the collaborative SVD model, as it will be weaker at picking up patterns. So, for that model, we will only keep movies with at least 22 (the median amount) ratings, as this will give the model a better chance of discovering patterns. As for the content-based filtering model, we can keep all of them, as the amount of ratings a movie has doesn't affect the model's performance. The hybrid model will in turn use only movies in both models as well.

To do this, we will keep only the movies with at least 22 ratings in the `ratings` file, for later use with the collaborative filtering model.

In [172…

```
# keeping only ratings for movies with 22+ ratings for the SVD model
popularly_rated_movies = movies[movies['num_ratings'] >= 22]['movieID']
ratings = ratings[ratings['movieID'].isin(popularly_rated_movies)]

# validating
print(f"Movies in content-based model: {movies.shape[0]}")
print(f"Movies in collaborative filtering model: {ratings['movieID'].nunique()}")
print(f"Ratings remaining: {ratings.shape[0]}")
print(f"Users remaining: {ratings['userID'].nunique()}")
```

```
Movies in content-based model: 9421

Movies in collaborative filtering model: 4732
Ratings remaining: 790173
Users remaining: 2113
```

As seen above, there are now:

- 9,421 movies in content-based model
- 4,732 movies in collaborative filtering (SVD) model
- 790,173 ratings remaining
- 2,113 users remaining

Also, in cutting all the movies with less than 22 ratings, we still kept 790,173 ratings out of the original 827,192 ratings. We are now ready to move on to building our recommendation systems. After all the cleaning, here is what the final `movies` and `ratings` files look like:

Movies Final Preview

In [173…
```
# movies
movies.head()
```

Out[173…

	movieID	title	year	title_year	num_ratings	genres	director	acto
0	1	Toy story	1995	Toy story (1995)	1263	Adventure Animation Children Comedy Fantasy	johnlasseter	tomhank timalle donrickle jimvarne wallace
1	2	Jumanji	1995	Jumanji (1995)	765	Adventure Children Fantasy	joejohnston	robinwilliam bonniehur kirstenduns bradleyp
2	4	Waiting to Exhale	1995	Waiting to Exhale (1995)	45	Comedy Drama Romance	forestwhitaker	whitneyhousto angelabasse lorettadevin lel
3	5	Father of the Bride Part II	1995	Father of the Bride Part II (1995)	225	Comedy	charlesshyer	stevemarti dianekeato martinsho kimberlywi
4	6	Heat	1995	Heat (1995)	586	Action Crime Thriller	michaelmann	alpacin robertdenir valkilme jonvoigh toms

Ratings Final Preview

In [174…
```
# ratings
ratings.head()
```

Out[174...

	userID	movieID	rating
1	75	32	4.5
2	75	110	4.0
3	75	160	2.0
4	75	163	4.0
5	75	165	4.5

5. Content-Based Recommender

Setup

Now that all the cleaning has been done, we can start building the first model, the content-based recommender system. This model will recommend movies based on the content of the movies. It will use each movies features to find other movies similar to a given movie. The first step is to combine all the features into one string for each movie, which will allow us to use a vectorizer. The vectorizer turns the string into a vector of numbers, which can then be used to calculate the similarity between movies. As stated in the features section at the beginning, the features we will be using for this model are:

- `genres`
- `director`
- `actors`
- `tags`

In [175...

```
# combining all into a string
movies['combined_features'] = (movies['genres'] + ' ' +
                               movies['director'] + ' ' +
                               movies['actors'] + ' ' +
                               movies['tags'])

# validating it works
print(f"Combined Features for {movies['title_year'][0]}:")
print(movies['combined_features'][0])
```

```
Combined Features for Toy story (1995):
Adventure Animation Children Comedy Fantasy johnlasseter tomhanks timallen donrickle
s jimvarney wallaceshawn funny timetravel timallen comedy fun fantasy pixar imdbtop2
50 disneyanimatedfeature tomhanks adventure animation wanttoseeagain witty classic c
lever family kidsmovie animated disney 3d erlendsdvds cganimation toys computeranima
tion usa warm heroicmission light almostfavorite want nationalfilmregistry unlikelyf
riendships pixaranimation avi tumeystoseeagain tumeysvhs rousing fanciful buy bright
toyscometolife daringrescues
```

Vectorization

Now that we have the `combined_features`, we can turn each movie's string into a vector of numbers. We will use the `TfidfVectorizer` rather than the `CountVectorizer` as it will place more emphasis on more unique words. For the Toy Story (1995) example, the word "Pixar" is more unique than the word "Animation", so it will be weighted more heavily in the vector. We will also use `stop_words='english'` to remove common words that don't add any value to the model, such as "the", "and", "of", etc.

In [176…
```
# putting each movie into the vectorizer
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(movies['combined_features'])

# checking shape
print(f"TF-IDF Matrix Shape: {tfidf_matrix.shape}")
```

```
TF-IDF Matrix Shape: (9421, 27453)
```

Function to Get Recommendations

Now that the TF-IDF matrix has been created, we can build out the content-based recommender function. It will take in the title of the movie and the number of recommendations to return, and will calculate the cosine similarity between the given movie and all other movies. Then it will sort them and return the top specified number of recommendations.

In [177…
```
def content_based_recommender(title, n=10):
    # getting index of movie
    matches = movies.index[movies['title_year'].str.lower() == title.lower()].tolis
    # if not in the dataset return an error
    if not matches:
        return f"Movie '{title}' not found in the dataset."
    movie_index = matches[0]
    # calculating cosine similarity
    scores = cosine_similarity(tfidf_matrix[movie_index], tfidf_matrix).flatten()
    # sorting + skipping the movie itself
    ranked = scores.argsort()[::-1]
    ranked = [i for i in ranked if i != movie_index][:n]
    # building the df of reccs
    recommendations = pd.DataFrame({
        'movieID': movies.iloc[ranked]['movieID'].values,
        'title_year': movies.iloc[ranked]['title_year'].values,
        'genres': movies.iloc[ranked]['genres'].values,
        'similarity_score': scores[ranked].round(3)
    })
    return recommendations
```

Testing on movies:

In [178…
```
content_based_recommender('The Dark Knight (2008)', 10)
```

Out[178...

	movieID	title_year	genres	similarity_score
0	33794	Batman Begins (2005)	Action Crime	0.299
1	56286	I'm Not There. (2007)	Drama Musical	0.252
2	48780	The Prestige (2006)	Drama Mystery Thriller	0.229
3	7372	Ned Kelly (2003)	Drama	0.218
4	42015	Casanova (2005)	Action Adventure Comedy Drama Romance	0.218
5	5611	The Four Feathers (2002)	Adventure War	0.190
6	4299	A Knight's Tale (2001)	Action Adventure Comedy	0.181
7	6681	All the Little Animals (1998)	Drama Thriller	0.175
8	49314	Harsh Times (2005)	Action Crime Drama	0.172
9	54997	3:10 to Yuma (2007)	Action Crime Drama Western	0.170

In [179...

```
content_based_recommender('Toy Story (1995)', 10)
```

Out[179...

	movieID	title_year	genres	similarity_score
0	8965	The Polar Express (2004)	Adventure Animation Children Fantasy IMAX	0.234
1	2355	A Bug's Life (1998)	Adventure Animation Children Comedy	0.213
2	2294	Antz (1998)	Adventure Animation Children Comedy Fantasy	0.198
3	8978	Christmas with the Kranks (2004)	Children Comedy	0.196
4	4774	Big Trouble (2002)	Comedy Crime	0.191
5	45517	Cars (2006)	Animation Children Comedy Fantasy	0.188
6	596	Pinocchio (1940)	Animation Children Fantasy Musical	0.187
7	1025	The Sword in the Stone (1963)	Animation Children Fantasy Musical	0.180
8	364	The Lion King (1994)	Adventure Animation Children Drama Musical	0.179
9	4016	The Emperor's New Groove (2000)	Adventure Animation Children Comedy Fantasy	0.179

Summary

As we can see above in our example run on 'The Dark Knight (2008)', the content-based recommender system returns 10 movies similar to the given movie. For example, Batman Begins (2005) is the prequel to The Dark Knight (2008), and The Prestige has both Christian Bale (who plays Batman in The Dark Knight) and Christopher Nolan (director of The Dark Knight). Also, for the example run on 'Toy Story (1995)', the recommendations are mostly animated family films, with a couple others including actors from Toy Story (1995). This shows that the content-based recommender system is able to find movies that are similar to the given movie based on the content of the movies.

6. Collaborative Recommender (SVD)

Setup

Now, we can build out our collaborative filtering recommender system. This model will recommend movies based on predictions of how a user would rate a movie by learning from the ratings of other users. It uses Singular Value Decomposition (SVD) to find latent factors that explain the ratings of users and movies. The first step is to create a user-item matrix, which will be used to train the SVD model.

In [180…

```
# building user-movie matrix (rows = users, columns = movies)
user_movie_ratings = ratings.pivot(index='userID', columns='movieID', values='ratin

# checking shape and empty values
print(f"User-Movie Matrix Shape: {user_movie_ratings.shape}")
print(f"Percent of cells empty: {user_movie_ratings.isna().sum().sum() / user_movie
```

```
User-Movie Matrix Shape: (2113, 4732)
Percent of cells empty: 92.1%
```

We can see that there are 2,113 `users` and 4,732 `movies` in the `user_movie_ratings` matrix. However, 92.1% of the cells are empty, which the SVD model will predict the missing values for.

For the function, we will follow this approach:

1. **Center Ratings**: doing so by subtracting each user's average rating. This will help in removing bias from harsh or generous raters, and we will fill empty cells with 0's for now.
2. **Run SVD**: running with `k` latent features.
3. **Rebuild Matrix**: rebuilding the user-item matrix with the predicted ratings filling in the empty cells.
4. **Adding Back User Averages**: adding back the user means to the predicted ratings

In [181…

```
def svd_ratings_prediction(user_movie_ratings, k=10):
    # step 1: normalizing ratings by average (using the mean)
    user_mean_rating = user_movie_ratings.mean(axis=1)
    normalized_ratings = user_movie_ratings.sub(user_mean_rating, axis=0)
    normalized_ratings = normalized_ratings.fillna(0)
    # converting to an array
    normalized_ratings = normalized_ratings.values

    # performing SVD
    U, sigma, Vt = svds(normalized_ratings, k=k)
    # sort in descending order (cause svds returns it in ascending order)
    descending_order = np.argsort(sigma)[::-1]
    sigma = sigma[descending_order]
    U = U[:, descending_order]
    Vt = Vt[descending_order, :]
    sigma_diagonal = np.diag(sigma)

    # adding back user averages
    normalized_predicted_ratings = np.dot(np.dot(U, sigma_diagonal), Vt)
    predicted_ratings = normalized_predicted_ratings + user_mean_rating.values.resh

    # keeping preds in the scale (0.5 - 5)
    predicted_ratings = np.clip(predicted_ratings, 0.5, 5)
    predicted_ratings_df = pd.DataFrame(predicted_ratings,
                                        index=user_movie_ratings.index,
                                        columns=user_movie_ratings.columns)
    return predicted_ratings_df
```

Choosing the Number of Latent Features

Now we need to choose `k`, the number of latent features in the SVD model. Too low of a `k` will lead to underfitting (not enough features to capture the complexity of the data), while too high of a `k` will lead to overfitting (too many features that capture noise rather than signal).

To choose `k` fairly, we will split the ratings into three parts:

- **Train (80%)**: used to build the SVD model
- **Validation (10%)**: used to compare different values of `k` and pick the best one
- **Test (10%)**: kept hidden until the very end, and only used once to report the final RMSE

The reason for the separate test set is that if we pick `k` using the same ratings we report the score on, the score would be a little too optimistic, since we basically chose the `k` that happened to do best on those exact ratings. Lower RMSE = better model, so we will choose the `k` with the lowest validation RMSE.

In [182…

```
# hiding 10% of the ratings for test
test = ratings.sample(frac=0.1, random_state=1)
remaining = ratings.drop(test.index)

# hiding another 10% of the remaining ratings for validation
```

```
validation = remaining.sample(frac=1/9, random_state=1)
train = remaining.drop(validation.index)

# buidling matrix from only the training data
train_matrix = train.pivot(index='userID', columns='movieID', values='rating')

# only keeping validation and test ratings that are in the training matrix (cause s
test = test[test['userID'].isin(train_matrix.index) & test['movieID'].isin(train_ma
validation = validation[validation['userID'].isin(train_matrix.index) & validation[

# validating
print(f"Training ratings: {train.shape[0]}")
print(f"Validation ratings: {validation.shape[0]}")
print(f"Test ratings: {test.shape[0]}")
```

```
Training ratings: 632139
Validation ratings: 79017
Test ratings: 79017
```

After the split, the ratings are now divided into:

- **Train (80%)**: 632,139 ratings
- **Validation (10%)**: 79,017 ratings
- **Test (10%)**: 79,017 ratings

In [183…

```
# predicting average ratings
user_avg = train.groupby('userID')['rating'].mean()
baseline_preds = validation['userID'].map(user_avg)
baseline_rmse = np.sqrt(np.mean((baseline_preds - validation['rating']) ** 2))
print(f"Baseline RMSE: {baseline_rmse:.4f}")
```

```
Baseline RMSE: 0.9194
```

The baseline RMSE on the validation set is 0.9194, meaning the predictions are off by about 0.9 stars on average if the model was to simply guess. The model should be able to perform better than this by learning patterns in the data. So now, we will test different values of `k` on the validation setto find the best one. The best `k` will be the one with the lowest RMSE as mentioned earlier.

In [184…

```
# testing diff values of k on validation set
k_values = [5, 10, 15, 20, 25, 30, 50, 100]
val_scores = {}
for k in k_values:
    predicted = svd_ratings_prediction(train_matrix, k=k)
    # predictions for hidden ratings in the validation set
    predictions = [predicted.at[u, m] for u, m in zip(validation['userID'], validat
    rmse = np.sqrt(np.mean((np.array(predictions) - validation['rating'].values) **
    val_scores[k] = rmse
    print(f"k={k}: RMSE={rmse:.4f}")

# picking k with lowest RMSE
best_k = min(val_scores, key=val_scores.get)
print(f"Best k: {best_k}")
```

```
k=5: RMSE=0.8463
k=10: RMSE=0.8394
k=15: RMSE=0.8367
k=20: RMSE=0.8359
k=25: RMSE=0.8364
k=30: RMSE=0.8367
k=50: RMSE=0.8416
k=100: RMSE=0.8575
Best k: 20
```

The validation RMSE is lowest when `k` is 20, with an RMSE of 0.8359. The scores for the `k` values between 15-30 are notably close, indicating that the model's performance is relatively stable within this range. Once the `k` values begin to exceed 50, we can see the RMSE starts climbing again, suggesting that the model is beginning to overfit the training data around those higher values.

Now that `k` has been chosen, we can use the test set. This is the first and only time the test ratings are used, so this is the honest estimate of how well the model predicts ratings it has never seen.

In [185…

```python
# training final model with best k (20)
final_preds = svd_ratings_prediction(train_matrix, k=best_k)

# SVD RMSE on test set
test_preds = [final_preds.at[u, m] for u, m in zip(test['userID'], test['movieID'])]
test_rmse = np.sqrt(np.mean((np.array(test_preds) - test['rating'].values) ** 2))

# baseline RMSE on test set (for comparison)
test_bl = test['userID'].map(user_avg)
test_bl_rmse = np.sqrt(np.mean((np.array(test_bl) - test['rating'].values) ** 2))

print(f"Test RMSE (SVD, k={best_k}): {test_rmse:.4f}")
print(f"Test RMSE (Baseline): {test_bl_rmse:.4f}")
```

```
Test RMSE (SVD, k=20): 0.8411
Test RMSE (Baseline): 0.9242
```

On the test set, the SVD model reached an RMSE of 0.8411, compared to 0.9242 for the baseline (about 9% better). The test RMSE is slightly higher than the validation RMSE (0.8359), which is expected since `k` was chosen specifically cause it performed best on the validation set. Since the test ratings had no part in choosing `k`, 0.8411 is the more honest measure of how the model performs on new ratings.

Function to Get Recommendations

In [186…

```python
def collaborative_recommender(user_id, n=10, k=best_k):
    # return error if user not in data
    if user_id not in user_movie_ratings.index:
        return f"User ID {user_id} not found in the dataset."

    # predicted ratings for all users and movies
    predicted_ratings_df = svd_ratings_prediction(user_movie_ratings, k=k)
```

```
    # only keeping movies the user hasn't rated yet
    unrated = user_movie_ratings.loc[user_id].isna()
    user_predictions = predicted_ratings_df.loc[user_id][unrated]

    # top n (10) highest predicted ratings
    top_movies = user_predictions.sort_values(ascending=False).head(n)

    # building df
    recommendations = top_movies.rename('predicted_rating').reset_index()
    recommendations = recommendations.merge(movies[['movieID', 'title_year', 'genre
    recommendations['predicted_rating'] = recommendations['predicted_rating'].round
    return recommendations[['movieID', 'title_year', 'genres', 'predicted_rating']]
```

Now we will test the recommender, by picking a user, looking at the movies they rated highest, and judging whether the recommendations match their tast.

In [187…
```
# picking user to test (just gonna use first one)
test_user = user_movie_ratings.index[0]

# pulling their top 10 highest rated movies
user_top = ratings[ratings['userID'] == test_user].sort_values('rating', ascending=
user_top = user_top.merge(movies[['movieID', 'title_year', 'genres']], on='movieID'
print(f"User {test_user}'s top 10 rated movies:")
user_top[['title_year', 'genres', 'rating']]
```

```
User 75's top 10 rated movies:
```

Out[187…

	title_year	genres	rating
0	Sin City (2005)	Action Crime Film_Noir Thriller	5.0
1	Pulp Fiction (1994)	Comedy Crime Drama	5.0
2	Twelve Monkeys (1995)	Sci_Fi Thriller	4.5
3	Last Man Standing (1996)	Action Crime Drama Thriller	4.5
4	Hostage (2005)	Action Crime Drama Thriller	4.5
5	The Last Boy Scout (1991)	Action Comedy Crime Drama Thriller	4.5
6	Fight Club (1999)	Action Crime Drama Thriller	4.5
7	South Park: Bigger Longer & Uncut (1999)	Animation Comedy Musical	4.5
8	The Matrix (1999)	Action Sci_Fi Thriller	4.5
9	The Fifth Element (1997)	Action Adventure Sci_Fi	4.5

Now we test the recommender on this user:

In [188…
```
collaborative_recommender(test_user, 10)
```

Out[188...

	movieID	title_year	genres	predicted_rating
0	6874	Kill Bill: Vol. 2 (2004)	Action Crime Thriller	3.796
1	47	Shichinin no samurai (1954)	Crime Horror Mystery Thriller	3.795
2	50	The Usual Suspects (1995)	Crime Mystery Thriller	3.773
3	4226	Memento (2000)	Crime Drama Mystery Thriller	3.765
4	1089	Reservoir Dogs (1992)	Crime Drama Thriller	3.743
5	318	The Shawshank Redemption (1994)	Drama	3.735
6	1196	Star Wars: Episode V - The Empire Strikes Back...	Action Adventure Sci_Fi	3.727
7	4011	Snatch. (2000)	Action Comedy Crime Thriller	3.722
8	3578	Gladiator (2000)	Action Adventure Drama	3.710
9	1200	Aliens (1986)	Action Horror Sci_Fi Thriller	3.709

Summary

User 75's highest rated movies are mostly crime and thriller films, like Sin City (2005), Fight Club (1999), and Pulp Fiction (1994), and the collaborative recommender system suggested movies similar. For example, it suggested Kill Bill: Vol. 2 (2004) and Reservoir Dogs (1992), both of which are Quentin Tarantino films like Pulp Fiction. It also suggested The Usual Suspects (1995), Memento (2000), and Snatch (2000), all of which are crime/thriller films. This shows that the collaborative recommender system is able to learn a user's taste purely from rating patterns, without needing to know anything about the content of the movies.

7. Hybrid Recommender

Combining the Models

Our last model is the hybrid recommender system, which combines the content-based and collaborative filtering approaches. It will use both the movie features and the ratings of users to recommend movies. First, we will find the 100 movies most similar to the given movie (uses content-based filtering), and then we will predict how the given user would rate each of those movies (uses collaborative filtering). We will then sort the movies by predicted rating to offer the recommendations.

In [189…

```
def hybrid_recommender(title, user_id, k=best_k, n=10):
    # if the user is not in the dataset, return an error
    if user_id not in user_movie_ratings.index:
        return f"User ID {user_id} not found in the dataset."

    # 100 most similar (from cbr)
    similar = content_based_recommender(title, 100)
    if isinstance(similar, str):
        return similar

    # predicting (from SVD)
    predicted_ratings_df = svd_ratings_prediction(user_movie_ratings, k=k)
    user_predictions = predicted_ratings_df.loc[user_id]

    # only keeping movies that are in SVD model and havent been rated
    already_rated = user_movie_ratings.loc[user_id].dropna().index
    similar = similar[similar['movieID'].isin(user_predictions.index) & ~similar['m

    # adding predicted ratings + ranking by it
    similar['predicted_rating'] = similar['movieID'].map(user_predictions).round(3)
    similar = similar.sort_values('predicted_rating', ascending=False).head(n)
    return similar.reset_index(drop=True)
```

Now we can test the hybrid recommender. We will use the same movie for two different users, and see if the recommendations are similar but in a different order based on each user's taste.

In [190…

```
# hybrid for the test_user (#75) again, and the movie 'The Dark Knight (2008)'
print(f"User {test_user}:")
hybrid_recommender('The Dark Knight (2008)', test_user, k=best_k, n=10)
```

```
User 75:
```

Out[190…

	movieID	title_year	genres	similarity_score	predicted_rating
0	33794	Batman Begins (2005)	Action Crime	0.299	3.668
1	5903	Equilibrium (2002)	Action Sci_Fi Thriller	0.116	3.625
2	48780	The Prestige (2006)	Drama Mystery Thriller	0.229	3.623
3	59315	Iron Man (2008)	Action Adventure Sci_Fi	0.099	3.603
4	48774	Children of Men (2006)	Action Adventure Drama Sci_Fi Thriller	0.085	3.592
5	8950	The Machinist (2004)	Drama Mystery Thriller	0.112	3.584
6	5617	Secretary (2002)	Comedy Drama Romance	0.144	3.582
7	3535	American Psycho (2000)	Comedy Crime Drama Horror Mystery Thriller	0.140	3.578
8	5611	The Four Feathers (2002)	Adventure War	0.190	3.570
9	5120	Sleuth (2007)	Comedy Mystery Thriller	0.169	3.566

Now we wil do the same movie but with a different user:

In [191…

```
other_user = user_movie_ratings.index[1]
print(f"User {other_user}:")
hybrid_recommender('The Dark Knight (2008)', other_user, k=best_k, n=10)
```

```
User 78:
```

Out[191...

	movieID	title_year	genres	similarity_score	predicted_rating
0	1303	The Man Who Would Be King (1975)	Adventure Drama	0.126	4.242
1	8950	The Machinist (2004)	Drama Mystery Thriller	0.112	4.212
2	5617	Secretary (2002)	Comedy Drama Romance	0.144	4.188
3	5120	Sleuth (2007)	Comedy Mystery Thriller	0.169	4.182
4	5938	Deathtrap (1982)	Comedy Crime Mystery Thriller	0.093	4.173
5	4103	Empire of the Sun (1987)	Drama War	0.124	4.169
6	3952	The Contender (2000)	Drama	0.089	4.167
7	521	Romeo Is Bleeding (1993)	Crime Thriller	0.100	4.166
8	2083	The Muppet Christmas Carol (1992)	Children Fantasy Musical	0.108	4.159
9	2579	Following (1998)	Crime Film_Noir Mystery Thriller	0.100	4.158

Summary

The hybrid recommender system is able to utilize both the content of the movie and the user's taste to recommend movies effectively. Every movie on the both lists connects to The Dark Knight (2008), as many of the actors are in both lists with matching genres as well. For example, actor Christian Bale is in The Dark Knight (2008), Batman Begins (2005), and The Machinist (2004). On top of this, both users still had personalized lists, as user 75 got more action/sci-fi movies whereas user 78 got more older/drama films, showing the hybrid recommender system is able to learn a user's taste and recommend movies accordingly.

8. Comparing the Models

So far, only the SVD model has a number attached to it (RMSE). The content-based and hybrid models were only judged by looking at their recommendations, which doesn't tell us much about how well they'd actually work. RMSE also can't be used for them, since they don't predict ratings.

To compare all three models fairly, we will use **hit rate @ 10**. For each user, we will:

1. Look at the movies they rated 4+ stars in the test set (their hidden "favorites")

2. Have each model recommend 10 movies the user hasn't rated in the training data
3. Count it as a **hit** if at least one of those 10 movies is one of their hidden favorites

The hit rate is the percentage of users who got at least one hit. We will also compare against a simple **popularity baseline**, which just recommends the most-rated movies to everyone. If a model can't beat that, it's not really learning anything about the user.

In [192…
```
# favorite = any test rating >= 4 stars
favorites = test[test['rating'] >= 4.0]

# picking 300 randoms
users_for_comp = favorites['userID'].drop_duplicates().sample(n=300, random_state=1

# validating
print(f"Favorite ratings in the test set: {favorites.shape[0]}")
print(f"Users with at least 1+ favorite: {favorites['userID'].nunique()}")
print(f"Users being evaluated: {len(users_for_comp)}")
```

```
Favorite ratings in the test set: 35274
Users with at least 1+ favorite: 2042
Users being evaluated: 300
```

Popularity Baseline

There are 35,274 favorite ratings (4+ stars) hidden in the test set, spread across 2,042 users. We randomly picked 300 of these users to evaluate the models on, since running the content-based and hybrid models for every user would take a long time.

We will start with the **popularity baseline**. For each user, it recommends the 10 most-rated movies (from the training data) that the user hasn't already rated.

In [193…
```
# movies sorted by how many ratings they have in training data
popular_movies = train['movieID'].value_counts().index
pop_hits = 0
for user in users_for_comp:
    # movies the user has already rated in the training data
    rated_movies = set(train[train['userID'] == user]['movieID'])
    # recommend the top 10 most popular movies the user hasn't rated
    recs = [m for m in popular_movies if m not in rated_movies][:10]
    # hidden favorites for the user
    user_favorites = set(favorites[favorites['userID'] == user]['movieID'])
    # hit if any are favs
    if len(set(recs) & user_favorites) > 0:
        pop_hits += 1

pop_hit_rate = pop_hits / len(users_for_comp)
print(f"Popularity baseline hit rate @ 10: {pop_hit_rate:.1%}")
```

```
Popularity baseline hit rate @ 10: 66.3%
```

The popularity baseline reached a hit rate of 66.3%, meaning about 2 out of every 3 users had at least one of their hidden favorites in the 10 most popular movies they hadn't rated

yet. This is a pretty high bar, since popular movies tend to be liked by a lot of people.

Collaborative (SVD) Model

Next we will use `final_preds` on the collaborative (SVD) model to see how well it performs compared to the popularity baseline. The model has not seen the hidden favorites since it was trained on the training data, not the test set.

In [194…

```
# same process as popularity baseline but now for svd
svd_hits = 0
for user in users_for_comp:
    seen = set(train[train['userID'] == user]['movieID'])
    user_preds = final_preds.loc[user]
    user_preds = user_preds[~user_preds.index.isin(seen)]
    recs = user_preds.sort_values(ascending=False).head(10).index
    user_favs = set(favorites[favorites['userID'] == user]['movieID'])
    if len(set(recs) & user_favs) > 0:
        svd_hits += 1

svd_hit_rate = svd_hits / len(users_for_comp)
print(f"Collaborative (SVD) hit rate @ 10: {svd_hit_rate:.1%}")
```

```
Collaborative (SVD) hit rate @ 10: 75.3%
```

The collaborative (SVD) model reached a hit rate of 75.3%, beating the popularity baseline by about 9 points. This shows the model is actually learning each user's taste from rating patterns, rather than simply recommending movies that are popular with everyone.

Content-Based Model

Next is the content-based model. Since it recommends movies based on a single movie instead of a user, we need to pick a movie for each user to base the recommendations on. We will use each user's highest rated movie from the training data. To keep the comparison fair, it can only recommend movies that are also in the SVD model, so all of the models are picking from the same pool of movies.

In [195…

```
# mapping each movie to its row in the tfidf matrix
movie_row = pd.Series(movies.index, index=movies['movieID'])

# same movies as svd model
svd_movies = set(final_preds.columns)

content_hits = 0
for user in users_for_comp:
    user_train = train[train['userID'] == user]
    seen = set(user_train['movieID'])
    # users highest
    top_movie = user_train.sort_values('rating', ascending = False)['movieID'].iloc
    # similarity between that movie + all other movies
    scores = cosine_similarity(tfidf_matrix[movie_row[top_movie]], tfidf_matrix).fl
```

```
    ranked_ids = movies['movieID'].values[scores.argsort()[::-1]]
    # top 10 movies similar in SVD pool and havent yet been rated
    recs = [m for m in ranked_ids if m in svd_movies and m not in seen][:10]
    user_favs = set(favorites[favorites['userID'] == user]['movieID'])
    if len(set(recs) & user_favs) > 0:
        content_hits += 1

content_hit_rate = content_hits / len(users_for_comp)
print(f"Content-based hit rate @ 10: {content_hit_rate:.1%}")
```

```
Content-based hit rate @ 10: 25.0%
```

The content-based model reached a hit rate of 25.0%, well below both the popularity baseline and the SVD model. This makes sense, since the content-based model only uses one movie per user to make its recommendations, while the SVD model learns from every rating the user has made. The content-based model is better at answering "what's similar to this movie" than "what will this user like overall."

Hybrid Model

Last is the hybrid model. Just like the hybrid function, it takes the 100 movies most similar to the user's highest rated movie, then ranks them by the user's predicted rating from the SVD model and keeps the top 10.

In [196…

```
hybrid_hits = 0
for user in users_for_comp:
    user_train = train[train['userID'] == user]
    seen = set(user_train['movieID'])
    # highest in training
    top_movie = user_train.sort_values('rating', ascending=False)['movieID'].iloc[0
    scores = cosine_similarity(tfidf_matrix[movie_row[top_movie]], tfidf_matrix).fl
    ranked_ids = movies['movieID'].values[scores.argsort()[::-1]]
    # 100 most similar movies in SVD that havent been rated
    candidates = [m for m in ranked_ids if m in svd_movies and m not in seen][:100]
    # ranking + keeping t10
    recs = final_preds.loc[user, candidates].sort_values(ascending=False).head(10).
    user_favs = set(favorites[favorites['userID'] == user]['movieID'])
    if len(set(recs) & user_favs) > 0:
        hybrid_hits += 1

hybrid_hit_rate = hybrid_hits / len(users_for_comp)
print(f"Hybrid hit rate @ 10: {hybrid_hit_rate:.1%}")
```

```
Hybrid hit rate @ 10: 53.3%
```

The hybrid model reached a hit rate of 53.3%, more than double the content-based model. Since the hybrid uses the exact same 100 similar movies as its starting point, this shows that using the SVD predictions to rank them makes a big difference. However, it still falls below the SVD model and the popularity baseline, since it can only recommend movies similar to one movie the user liked, which limits how many of their favorites it wil find.

Summary

Now we can show all the hit rates in a table to compare/contrast the performance of the different recommendation models.

In [197…

```
# putting all the results into one table
results = pd.DataFrame({
    'model': ['Popularity Baseline', 'Content-Based', 'Collaborative (SVD)', 'Hybri
    'hit_rate_at_10': [pop_hit_rate, content_hit_rate, svd_hit_rate, hybrid_hit_rat
})
results['hit_rate_at_10'] = (results['hit_rate_at_10'] * 100).round(1)
results.sort_values('hit_rate_at_10', ascending=False).reset_index(drop=True)
```

Out[197…

	model	hit_rate_at_10
0	Collaborative (SVD)	75.3
1	Popularity Baseline	66.3
2	Hybrid	53.3
3	Content-Based	25.0

The collaborative (SVD) model performed the best by a clear margin, and it was the only model to beat the popularity baseline. The content-based and hybrid models scored lower, but that is partly because of how they were tested. Both are built to answer "what's similar to this movie," so they were only given one movie per user, while the SVD model and popularity baseline didn't have that limit.

The content-based and hybrid models are still useful when a user wants something similar to a specific movie, which the SVD model can't do. But for recommending movies based on a user's overall taste, the SVD model is the strongest of the three.

9. Conclusion and Limitations

Conclusion

In this project, we built three different types of movie recommender systems using the HetRec 2011 MovieLens dataset:

- The **content-based filtering** model, which recommends movies based on the content and features of the movies. We used `genres`, `director`, `actors`, and `tags` to find similar movies. It did well at finding similar movies, for example recommending other animated family films when prompted with 'Toy Story (1995)'. However, it had the lowest hit rate @ 10 (25.0%), since it only uses one movie at a time rather than a user's full rating history.

- The **collaborative filtering (SVD)** model, which recommends movies based on the ratings of other users. Using `k` = 20 latent features (chosen on a validation set), it reached a test RMSE of 0.8411, about 9% better than the baseline of 0.9242. It also had the highest hit rate @ 10 (75.3%) and was the only model to beat the popularity baseline (66.3%). Its recommendations matched the test user's taste well, for instance recommending other crime/thriller films to a user who rated 'Pulp Fiction (1994)' highly.
- The **hybrid filtering** model, which combines both content-based and collaborative filtering approaches. It recommended films with Christian Bale when prompted with 'The Dark Knight (2008)' for two different users, but in different orders based on each user's taste. It reached a hit rate @ 10 of 53.3%, more than double the content-based model, showing that ranking similar movies by SVD predictions makes a big difference.

A large part of the project was data cleaning and preprocessing, where we found and fixed several pre-existing issues within the dataset. This included duplicate movies, inconsistent director information, messy tags, etc. After cleaning, we built the three recommender systems and evaluated them using a train/validation/test split, so the final results were measured on ratings the models had never seen.

Limitations

- It seems that the cast and the director of a movie may dominate the content model, as shared actors sometimes outweigh other features more than they should (ex. unrelated movies to Toy Story (1995) were recommended because they shared the same actor, Tim Allen). This could be fixed by weighting the features differently (such as genres).
- Tag weights were discarded as all tags counted equally in the content-based filtering model. However, they could be used to weight the tags differently, as some tags may be more useful/important than others.
- Mislabeled movies were dropped rather than corrected, so some movies that could have been useful were not included in the models. Some mislabels also weren't caught at all. For example, movieID 47 shows up as 'Shichinin no samurai (1954)' (Seven Samurai), but its genres (Crime Horror Mystery Thriller) match Se7en (1995), which is what that ID actually is in MovieLens. This could be fixed by manually correcting the mislabeled movies, or by using a different dataset.
- The SVD model is limited to only movies with 22+ ratings. This is important for the SVD model to learn patterns, but it also means that the hybrid model is also only limited to movies with 22+ ratings. This could be fixed by using a dataset containing movies with more ratings on average.
- The hit rate @ 10 was only measured on 300 randomly sampled users to keep the run time down. Using all 2,042 users with hidden favorites would give a more reliable result.
- In the hit rate comparison, the content-based and hybrid models were only given each user's single highest rated movie, which puts them at a disadvantage compared to the SVD model. Using several of a user's top movies could give a fairer comparison.